

Отчёт по лабораторной работе №5

Сети Петри. Задача «Обедающие философы»

Черная София Витальевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Сети Петри	7
3.2	Свойства сетей Петри	8
3.3	Задача «Обедающие философы»	8
3.4	Методы моделирования	8
4	Выполнение лабораторной работы	10
4.1	Создание модуля сети Петри	10
4.2	Создание основного скрипта	11
4.3	Создание анимации	14
4.4	Создание отчета с графиками	16
4.5	Литературное программирование	18
5	Код и анализ	19
5.1	Код LITdining_philosophers.jl	19
5.2	Код LITdining_philosophers_animation.jl	21
5.3	Код LITdining_philosophers_report.jl	23
6	Вывод	25
	Список литературы	26

Список иллюстраций

4.1	Создание файла модуля	10
4.2	Код модуля DiningPhilosophers.jl	10
4.3	Создаю файл dining_philosophers.jl	11
4.4	Код dining_philosophers.jl	11
4.5	Установка пакета OrdinaryDiffEq	12
4.6	Запуск dining_philosophers.jl	12
4.7	Графики классической сети	13
4.8	Графики сети с арбитром	14
4.9	Создание файла анимации	14
4.10	Код анимации	15
4.11	Запуск dining_philosophers_animation	15
4.12	Визуализация графика philosophers_simulation.gif в начальное время 0	16
4.13	Создание файла отчета	16
4.14	Код отчёта	17
4.15	Запуск dining_philosophers_report.jl	17
4.16	Сравнительный график	17

Список таблиц

1 Цель работы

Освоение аппарата сетей Петри на примере классической задачи синхронизации «Обедающие философы». Изучение базовых элементов сетей Петри (позиции, переходы, фишки, дуги), правил срабатывания переходов и свойств систем (достижимость, живость, ограниченность). Приобретение навыков выявления и предотвращения взаимных блокировок (deadlock) в параллельных системах. Реализация детерминированного и стохастического имитационного моделирования с использованием языка Julia и пакетов DifferentialEquations.jl, Plots.jl.

2 Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код модели сети Петри для задачи «Обедающие философы».
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:
 - чистый код;
 - Jupyter Notebook;
 - документацию в формате Quarto.
6. Выполнить код из Jupyter Notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - Jupyter Notebook;
 - документацию в формате Quarto.
10. Выполнить код из Jupyter Notebook с параметрами.
11. Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

3.1 Сети Петри

Сети Петри — это математический аппарат для моделирования дискретных параллельных систем.

Сети Петри были предложены Карлом Адамом Петри в 1962 году [2]. Задача «Обедающие философы» была сформулирована Эдсгером Дейкстрой [1]. Методические указания к лабораторной работе [3].

Сеть Петри состоит из четырёх базовых элементов:

- **Позиции** (кружки) — описывают состояния системы или наличие ресурсов.
- **Фишки** (точки внутри позиций) — количество ресурсов или кратность выполнения условия.
- **Переходы** (прямоугольники) — описывают события или действия.
- **Дуги** (стрелки) — соединяют позиции с переходами (и наоборот), показывают, сколько фишек забирается или добавляется.

Правило срабатывания: переход разрешён, если во всех его входных позициях есть хотя бы столько фишек, сколько требует кратность входной дуги. При срабатывании фишки забираются из входов и добавляются в выходы.

3.2 Свойства сетей Петри

- **Ограниченность** — количество фишек в позициях не превышает заданного предела.
- **Живость** — любой переход может сработать хотя бы один раз.
- **Достижимость** — можно ли из начальной маркировки попасть в заданную.
- **Deadlock** (взаимная блокировка) — ни один переход не может сработать, система замирает.

3.3 Задача «Обедающие философы»

Классическая задача синхронизации, сформулированная Эдсгером Дейкстрой в 1965 году.

Постановка: За круглым столом сидят 5 философов. Перед каждым — тарелка с едой. Между каждыми двумя соседями лежит одна вилка. Всего вилок 5. Каждый философ может думать, быть голодным или есть. Чтобы поесть, нужны две вилки — левая и правая. Поев, философ кладёт вилки обратно.

Проблема: Если все философы одновременно возьмут левую вилку, каждый будет ждать правую, которая уже занята соседом. Возникает **deadlock** — никто не может есть, система встаёт.

Решение: Вводится арбитр (официант), который разрешает есть не более чем $N-1$ философам одновременно.

3.4 Методы моделирования

- **Детерминированное моделирование** — система обыкновенных дифференциальных уравнений, решение методом Tsit5.

- **Стохастическое моделирование** — алгоритм Гиллеспи, случайные времена между событиями.

4 Выполнение лабораторной работы

4.1 Создание модуля сети Петри

Создаю файл `src/DiningPhilosophers.jl` с определением структуры `PetriNet`, функций построения сетей и симуляции (рис. 4.1).

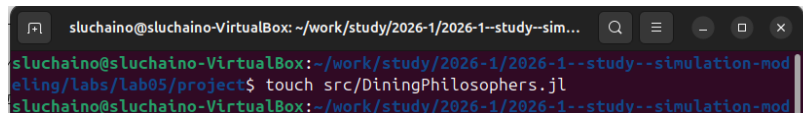


Рисунок 4.1: Создание файла модуля

Код модуля содержит определение структуры `PetriNet` (позиции, переходы, матрица инцидентности), функции `build_classical_network` и `build_arbiter_network` для построения сетей, а также `simulate_ode` и `simulate_stochastic` для двух типов моделирования (рис. 4.2).

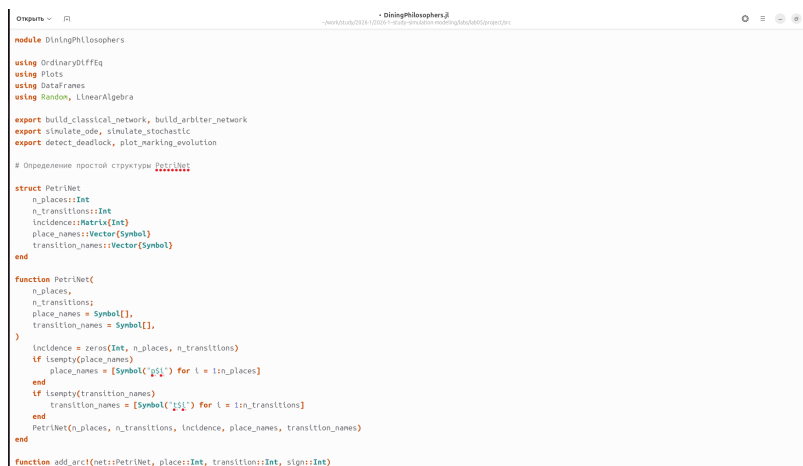


Рисунок 4.2: Код модуля `DiningPhilosophers.jl`

4.2 Создание основного скрипта

Создаю файл `scripts/dining_philosophers.jl` для запуска базовых экспериментов (рис. 4.3).

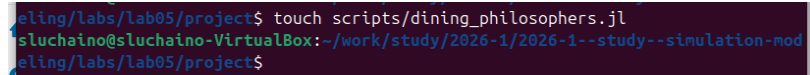


Рисунок 4.3: Создаю файл `dining_philosophers.jl`

Ввожу код для классической сети и сети с арбитром (рис. 4.4).



```
using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots

N = 5
tmax = 50.0

println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))

println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))
```

Рисунок 4.4: Код `dining_philosophers.jl`

Устанавливаю необходимые пакеты: `OrdinaryDiffEq` для решения ОДУ (рис. 4.5).

```
sluchaino@sluchaino-VirtualBox: /work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project$ julia
Documentation: https://docs.julialang.org
Type "?" for help, "}" for Pkg help.
Version 1.12.5 (2026-02-09)
Official https://julialang.org release

julia> import Pkg
julia> Pkg.add("OrdinaryDiffEq")
Resolving package versions...
Updating ~/julia/environments/v1.12/Project.toml
+ OrdinaryDiffEq v6.105.0
Manifest No packages added to or removed from ~/julia/environments/v1.12/Manifest.toml
```

Рисунок 4.5: Установка пакета OrdinaryDiffEq

Запускаю основной скрипт `dining_philosophers.jl` (рис. 4.6).

```
sluchaino@sluchaino-VirtualBox: /work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project$ julia --project=. scripts/dining_philosophers.jl
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
```

Рисунок 4.6: Запуск `dining_philosophers.jl`

Графики эволюции маркировки для классической сети. На четырех панелях показаны состояния Think, Hungry, Eat и Fork для каждого философа (рис. 4.7).

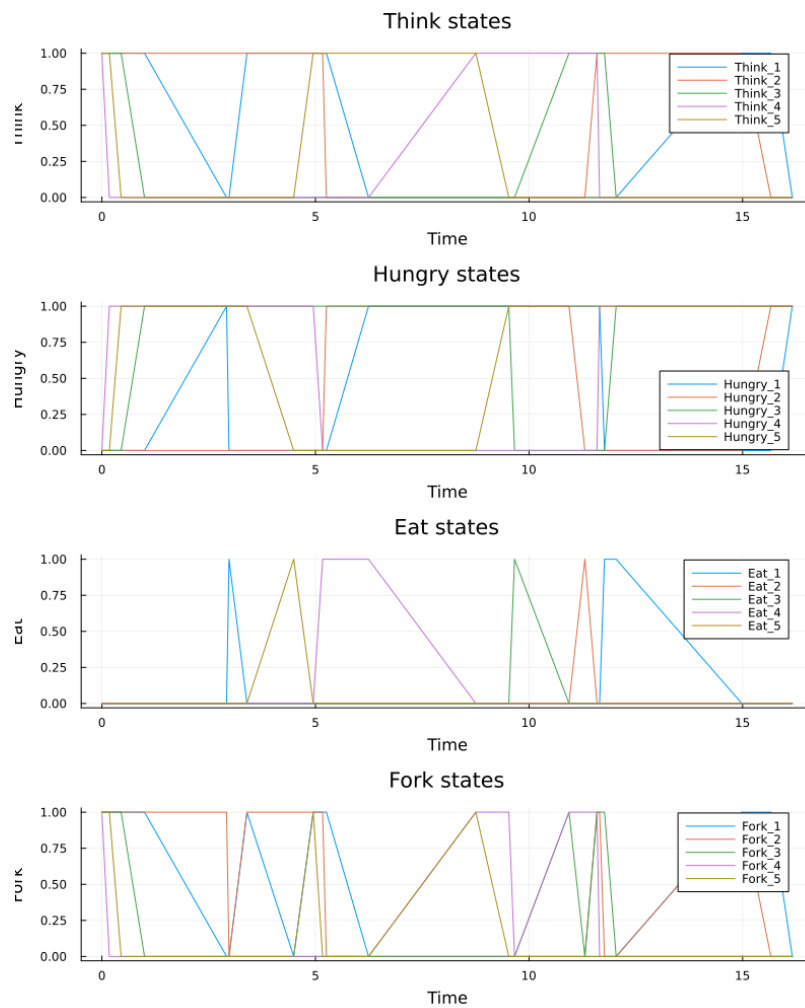


Рисунок 4.7: Графики классической сети

Графики эволюции маркировки для сети с арбитром. Видно, что система не встаёт в deadlock. А так же скачки стали более частыми (рис. 4.8).

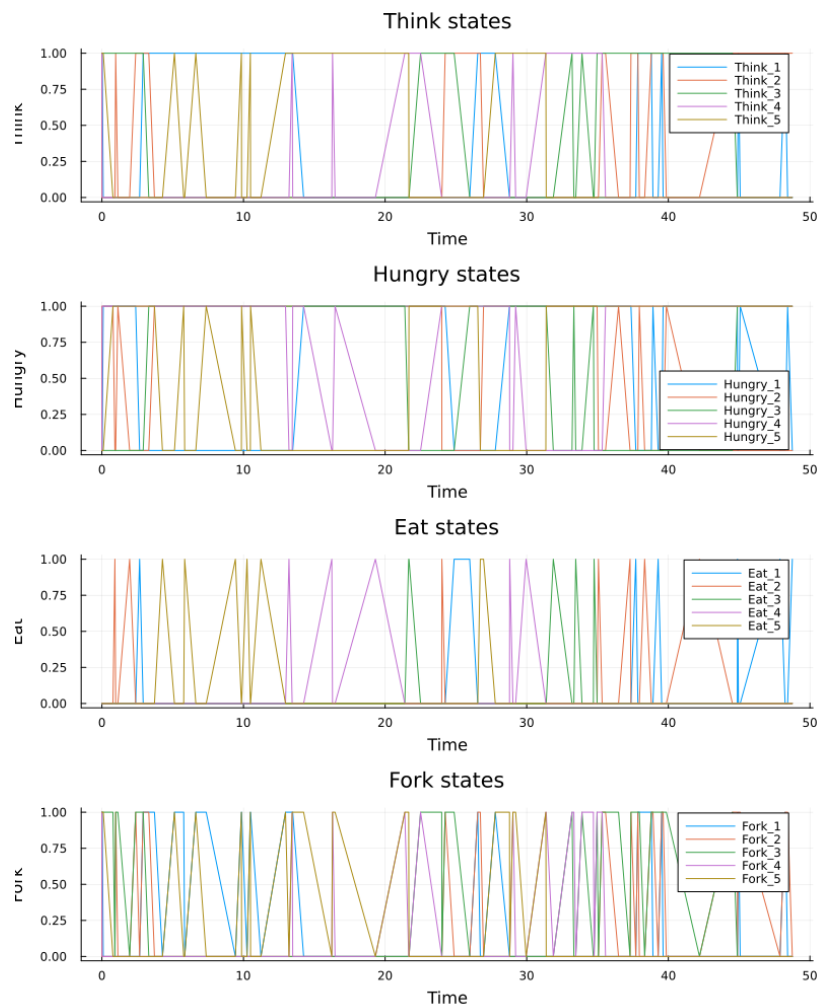


Рисунок 4.8: Графики сети с арбитром

4.3 Создание анимации

Создаю файл `scripts/dining_pholosophers_animation.jl` для анимации процесса (рис. 4.9).

```
sluchaino@sluchaino-VirtualBox: ~/work/study/2026-1/2026-1--study--sim...
sluchaino@sluchaino-VirtualBox:~/work/study/2026-1/2026-1--study--simulation-mod
eling/labs/lab05/project$ touch scripts/dining_pholosophers_animation.jl
sluchaino@sluchaino-VirtualBox:~/work/study/2026-1/2026-1--study--simulation-mod
eling/labs/lab05/project$
```

Рисунок 4.9: Создание файла анимации

Ввожу код анимации с использованием макроса `@animate` (рис. 4.10).

```
using DrWatson
@quickactivate "project"
include(scrdir("DiningPhilosophers.jl"))
using DiningPhilosophers
using Plots, Random

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)

Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u),
        u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = $(round(row.time, digits=2))",
    )
    xticks!(1:length(u), string.(names), rotation = 45)
end

gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
println("Анимация сохранена в plots/philosophers_simulation.gif")
```

Рисунок 4.10: Код анимации

Запускаю скрипт анимации (рис. 4.11).

```
jl@sluclidean@sluclidean-VirtualBox: /home/sluc.../simulation-modeling/labs/lab05/project$ julia --project=. scripts/dining_philosophers_animation.jl
could not create decoration from factory! Running with no decorations.
[ Info: Saved animation to /home/sluc.../work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project/plots/philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
```

Рисунок 4.11: Запуск dining_philosophers_animation

Анимация сохранена в файл `philosophers_simulation.gif` и наглядно показывает, как фишки перемещаются между позициями в процессе работы сети. (рис. 4.12).

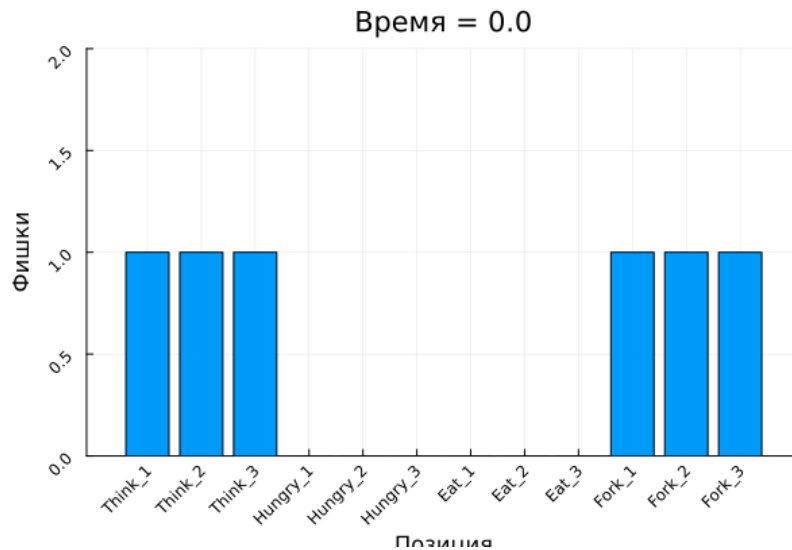


Рисунок 4.12: Визуализация графика philosophers_simulation.gif в начальное время 0

4.4 Создание отчета с графиками

Создаю файл `scripts/dining_philosophers_report.jl` для построения сравнительного графика (рис. 4.13).

```
luchaino@luchaino-VirtualBox: /work/study/2024-12/2024-12-study/simulation-modeling/lec1/lecs/scripts$ touch scripts/dining_philosophers_report.jl
luchaino@luchaino-VirtualBox: /work/study/2024-12/2024-12-study/simulation-modeling/lec1/lecs/scripts$
```

Рисунок 4.13: Создание файла отчета

Ввожу код для загрузки CSV-файла и построения графиков «Ест» для обеих сетей (рис. 4.14).


```

using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots

df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
N = 5

# Столбцы для состояния "Ест"
eat_cols = [Symbol("eat_$(i)") for i = 1:N]

p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = ["φ $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)
p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = ["φ $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Сеть с арбитром",
)
p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotsdir("final_report.png"))

println("Отчёт сохранён в plots/final_report.png")

```

Рисунок 4.14: Код отчёта

Запускаю скрипт отчёта (рис. 4.15).

```

slush@slushbox:~/VirtualBox/Projects/2022-2023/1-semester/algorithm-modeling/4th/2022/01/01 $ julia --project= scripts/dining_philosophers_report.jl
Отчёт сохранён в plots/final_report.png
Could not create decoration from factory! Running with no decorations.

```

Рисунок 4.15: Запуск dining_philosophers_report.jl

Сравнительный график сохранён в final_report.png (рис. 4.16).

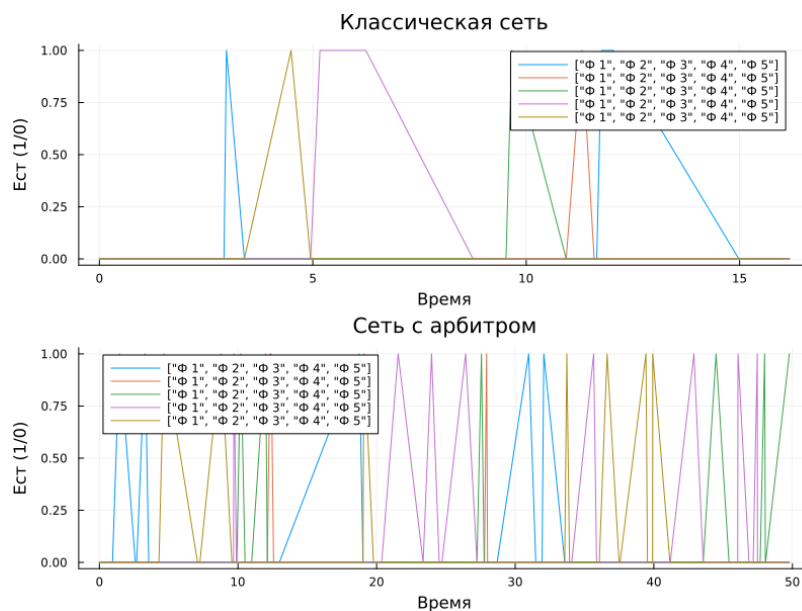


Рисунок 4.16: Сравнительный график

4.5 Литературное программирование

Все скрипты преобразованы в литературный стиль с подробными комментариями. Через `tangle.jl` из литературных скриптов сгенерированы чистый код, Jupyter Notebook и Quarto-документы.

5 Код и анализ

5.1 Код LITdining_philosophers.jl

```
# # Simulation of the Dining Philosophers problem using Petri nets

# This script models the classic synchronization problem using Petri
# Two models are compared: classical (leads to deadlock) and modified

# ## Environment setup

using DrWatson
@quickactivate "project"

include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers

using DataFrames, CSV, Plots, Random

# ## Simulation parameters

N = 5
```

```

tmax = 50.0

# ## Classical network (without arbiter)

println("=== Classical network (without arbiter) ===")

net_classic, u0_classic, _ = build_classical_network(N)

Random.seed!(456)

df_classic = simulate_stochastic(net_classic, u0_classic, tmax)

CSV.write(datadir("dining_classic.csv"), df_classic)

dead = detect_deadlock(df_classic, net_classic)
println("Deadlock detected: $dead")

plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))

# ## Network with arbiter

println("\n=== Network with arbiter ===")

net_arb, u0_arb, _ = build_arbiter_network(N)

df_arb = simulate_stochastic(net_arb, u0_arb, tmax)

```

```

CSV.write(datadir("dining_arbiter.csv"), df_arb)

dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock detected: $dead_arb")

plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))

```

5.2 Код LITdining_philosophers_animation.jl

```

# # Animation of Petri net for the Dining Philosophers problem

# This script creates an animation showing how the Petri net marking
# The animation clearly demonstrates the occurrence of deadlock.

# ## Environment setup

using DrWatson
@quickactivate "project"

include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers

using Plots, Random

# ## Simulation parameters

```

```

N = 3
tmax = 30.0

net, u0, names = build_classical_network(N)

Random.seed!(123)

df = simulate_stochastic(net, u0, tmax)

# ## Creating animation

anim = @animate for row in eachrow(df)

    u = [row[col] for col in propertynames(row) if col != :time]

    bar(
        1:length(u),
        u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Place",
        ylabel = "Tokens",
        title = "Time = $(round(row.time, digits=2))",
    )

    xticks!(1:length(u), string.(names), rotation = 45)
end

```

```
# ## Saving animation

gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)

println("Animation saved to plots/philosophers_simulation.gif")
```

5.3 Код LITdining_philosophers_report.jl

```
# # Comparative analysis of classical network and network with arbiter

# This script loads results from two simulations (classical and arbiter)
# and builds a comparative plot of the "Eating" state for all five philosophers.
# The plot clearly shows the difference: classical network ends in a deadlock
# while the network with arbiter does not.

# ## Loading data

using DrWatson
@quickactivate "project"

using DataFrames, CSV, Plots

df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)

N = 5
```

```

eat_cols = [Symbol("Eat_$i") for i in 1:N]

# ## Plotting

p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = ["P$i" for i in 1:N],
    xlabel = "Time",
    ylabel = "Eating (1/0)",
    title = "Classical network",
)

p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = ["P$i" for i in 1:N],
    xlabel = "Time",
    ylabel = "Eating (1/0)",
    title = "Network with arbiter",
)

p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))

savefig(plotsdir("final_report.png"))

println("Report saved to plots/final_report.png")

```


6 Вывод

В ходе выполнения лабораторной работы был освоен аппарат сетей Петри на примере классической задачи «Обедающие философы». Построены две модели сети Петри: классическая (с deadlock) и модифицированная с арбитром (без deadlock). Проведены детерминированное и стохастическое имитационное моделирование. Построены графики эволюции маркировки и создана анимация работы сети. Выполнено преобразование кода в литературный стиль и генерация производных форматов. Полученные результаты наглядно демонстрируют проблему взаимной блокировки в параллельных системах и способы её предотвращения.

Список литературы

1. *Dijkstra E. W.* Solution of a problem in concurrent programming control // Communications of the ACM. — 1965. — Т. 8, № 9. — С. 569.
2. *Petri C. A.* Kommunikation mit Automaten. — Universität Bonn, 1962.
3. *Королькова А. В., Кулябов Д. С.* Лабораторная работа №5: Сети Петри. Задача «Обедающие философы» / Российский университет дружбы народов. — 2026. — URL: <https://esystem.rudn.ru/mod/resource/view.php?id=1356135>.